RESEARCH Open Access

https://doi.org/10.1186/s40537-026-xxxxx-x

AST-Driven Empirical Benchmarking of SQL Query Complexity Across Serverless Cloud, Vectorized CPU (C++/Rust), and GPU-Accelerated OLAP Engines

Preecha Noiumkar[1*]

[1]Department of Information Technology, Faculty of Informatics, Mahasarakham University, Maha Sarakham 44150, Thailand.

[*]Correspondence: preecha.n@msu.ac.th

Abstract

Background: Evaluating modern analytical database architectures requires decoupling fixed distributed cloud orchestration and PCIe host-to-device transfer overheads from intrinsic structural query complexity and predicate selectivity. Prior OLAP benchmarks often rely on narrow engine pairs and unstratified query sets where memory layout and operator depth are confounded.

Methodology: We present a reproducible, cost-safe empirical benchmarking framework evaluating SQL query performance across a factorial benchmark of 75 analytical queries instantiated from 25 canonical seed templates across five structural complexity levels (L1–L5) and three selectivity perturbation horizons (V1–V3). Query complexity is formally quantified via static Abstract Syntax Tree (AST) profiling—capturing tree depth, operator node count, multi-way join fan-out, window frame specifications, and semi-structured array unnesting (`UNNEST`)—into a Composite Structural Complexity Index (CSCI). We evaluate six distinct software engines spanning four computing paradigms across multi-scale TPC-H datasets (SF $\in \{1, 10, 100\}$ GB) and the official 22-query TPC-H suite ($Q1$–$Q22$): (1) Google BigQuery (serverless cloud OLAP), (2) DuckDB (C++ morsel-driven vectorized DBMS), (3) Polars Lazy SQL (Rust Apache Arrow streaming engine), (4) Apache DataFusion (Rust Arrow physical query planner), (5) SQLite 3 (traditional C B-tree row-store baseline), and (6) a SIMT GPU-accelerated relational tensor engine on an NVIDIA GeForce RTX 2050 GPU (4.0 GB VRAM).

Results: Omnibus non-parametric Friedman testing confirms highly significant performance stratification across the six engines ($\chi^2_F(5) = 342.8, p < 10^{-16}$, Kendall's concordance $W = 0.914$). Columnar vectorization (DuckDB, Polars, and DataFusion) outperforms the row-store baseline (5071.1 ms, 0.064 GB/s) by $27.6\times$–$76.2\times$, elevating effective scan throughput to 2.63–6.98 GB/s. On Google BigQuery, storage clustering reduces mean wall-clock latency from 1412.6 ms to 745.4 ms ($p < 0.001$, slot concurrency multiplier $\mathscr{M}_{\text{slot}} = 6.03\times$) and prunes scanned I/O by 78.4% under high-selectivity workloads (V1), reducing cloud FinOps on-demand query expenditure by \$1.47 USD per million queries. Across in-memory engines, Rust Arrow-native engines (Polars at 66.52 ms and DataFusion at 88.20 ms) excel in zero-copy array unnesting (L5: 16.46–25.51 ms, peaking at 18.87 GB/s), whereas DuckDB outperforms DataFusion by $2.47\times$ in complex multi-axis window functions (L4: 78.16 ms vs. 193.20 ms). Finally, hardware-timed CUDA decomposition reveals that while a 55.5% PCIe overhead fraction (Φ_{PCIe}) makes cold-start GPU execution slower than CPU engines in L1 scans (55.02 ms vs. 50.84 ms), GPU acceleration achieves an overall 22.25 ms warm-VRAM mean latency (17.99 GB/s) and dominates L4 window sorting (19.37 ms cold, 9.94 ms warm at 27.95 GB/s).

Conclusion: Scaling to SF $= 100$ GB (8.66×10^8 rows) triggers up to 43.80 GB of out-of-core NVMe disk spilling in multi-way joins, widening DuckDB's native columnar advantage over ZSTD Parquet to $2.12\times$, while our batched row-group PCIe streaming engine bounds RTX 2050 VRAM usage to ≤ 0.149 GB without out-of-memory failures. Finally, a learned AST-aware query router evaluated via 5-fold seed-cluster cross-validation achieves a $4.07\times$ speedup over single-engine DuckDB (74.7% exact optimal match, 81.3% within-5 ms optimal). All 75 SQL workloads, AST analyzers, and SHA-256 telemetry logs are publicly available at https://github.com/preecha-msu/bigquery-olap-benchmark.

Keywords: Abstract syntax tree (AST), Apache Arrow, Apache DataFusion, Columnar storage, CUDA GPU acceleration, DuckDB, Google BigQuery, OLAP benchmark, Polars, SQL complexity.

1 Introduction

The rapid evolution of analytical data infrastructures has diversified enterprise architectures across four distinct computing paradigms: elastic serverless cloud data warehouses such as Google BigQuery [1, 2, 11], in-process C++ vectorized analytical database systems such as DuckDB [3], Rust-based Apache Arrow streaming query engines such as Polars and Apache DataFusion, and SIMT GPU-accelerated relational engines, alongside distributed systems like ClickHouse [4] and Trino [5]. While cloud-native disaggregated storage-compute architectures provide massive horizontal slot concurrency for petabyte-scale workloads [13, 15], they introduce non-trivial stateless scheduling and distributed shuffle overheads on moderate-scale datasets [9, 10, 11]. Concurrently, modern local hardware offers a complex trade-off between CPU SIMD vectorization over Apache Arrow buffers and discrete GPU offloading across the PCIe bus [12].

A fundamental methodological limitation in prior big data and OLAP comparisons [14, 15] is the reliance on narrow engine pairs and unstratified query sets where predicate selectivity, memory layout (row-oriented vs. columnar Parquet vs. Arrow memory), and structural operator depth are confounded. To rigorously decouple engine orchestration and PCIe transfer overheads from intrinsic SQL complexity, we construct a 75-query factorial benchmark derived from $N_{\text{seeds}} = 25$ canonical seed templates spanning five structural complexity tiers (L1 through L5) and three selectivity variations (V1 through V3), grounded in static Abstract Syntax Tree (AST) parsing via `SQLGlot` [6], and evaluate it across six heterogeneous software engines.

2 Related Work

Disaggregated cloud data warehouses pioneered by Google Dremel/BigQuery [1, 2] and distributed query engines such as Presto/Trino [5], ClickHouse [4], and Apache Spark [14] separate compute slots from durable object storage. Recent studies in serverless data pipelines [13] and in-situ serverless query processing such as FunDa [11] highlight the critical challenge of amortizing cold-start scheduling overhead against parallel slot elasticity. Conversely, embeddable C++ vectorized engines such as DuckDB [3] process columnar data chunks (Row Groups) using morsel-driven parallelism, whereas next-generation Rust engines (Polars and Apache DataFusion) execute zero-copy SIMD operations directly over standardized Apache Arrow columnar arrays. Concurrently, recent surveys on query execution optimization [12] emphasize the growing synergy between physical storage sorting, learned/rule-based optimizers [9], and GPU hardware acceleration. In parallel, modern SQL evaluation benchmarks, including BIRD [7] and Spider 2.0 [8], incorporate execution efficiency via the Valid Efficiency Score (VES), while AST transpilers such as `SQLGlot` [6] enable deterministic cross-dialect structural inspection.

3 Methodology and Six-Software Benchmark Design

3.1 Five-Tier SQL Structural Complexity and 25 Canonical Seeds

As illustrated in the end-to-end experimental workflow (Fig. 1), each benchmark query $q \in \mathcal{Q}$ ($|\mathcal{Q}| = 75$) is parsed into an Abstract Syntax Tree (AST) using `SQLGlot` [6] to extract structural complexity features: tree depth $D(q)$, total AST nodes $N(q)$, relational join count $J(q)$, nested CTE/subquery depth $S(q)$, window specifications $W(q)$, conditional branches $C(q)$, and repeated field unnesting operators $U(q)$. We define the Composite Structural Complexity Index (CSCI) as:

$$\text{CSCI}(q) = 1.5D(q) + 1.0\log_2 N(q) + 2.5J(q) + 3.0S(q) + 3.5W(q) + 4.0U(q) + 1.5C(q) \tag{1}$$

Across the TPC-H schema (SF = 1 GB, 8 relational tables totaling 8.66×10^6 rows), we design 5 canonical seeds per tier ($5 \times 5 = 25$ independent seeds):

1. **L1 (Scan, SARGability, & Monotonic Conditional Branching):** Single-table columnar scans with multi-branch `CASE WHEN` expressions and distinct cardinality aggregations ($D = 9.0, J = 0.0, \text{CSCI}_{v2} = 27.9$).
2. **L2 (Multi-Way Star, Snowflake, & Bilateral Joins):** 4-way to 8-way relational joins, composite `partsupp` joins, and non-equi late-delivery predicates ($D = 9.2, J = 4.8, \text{CSCI}_{v2} = 36.2$).
3. **L3 (Multi-Stage CTE Pipelines, Null-Safe Anti-Joins, & One-Grain-Per-CTE):** 3-stage to 4-stage Common Table Expressions enforcing pre-aggregated grain isolation and null-safe anti-joins ($D = 10.8, J = 2.6, \text{CSCI}_{v2} = 41.4$).

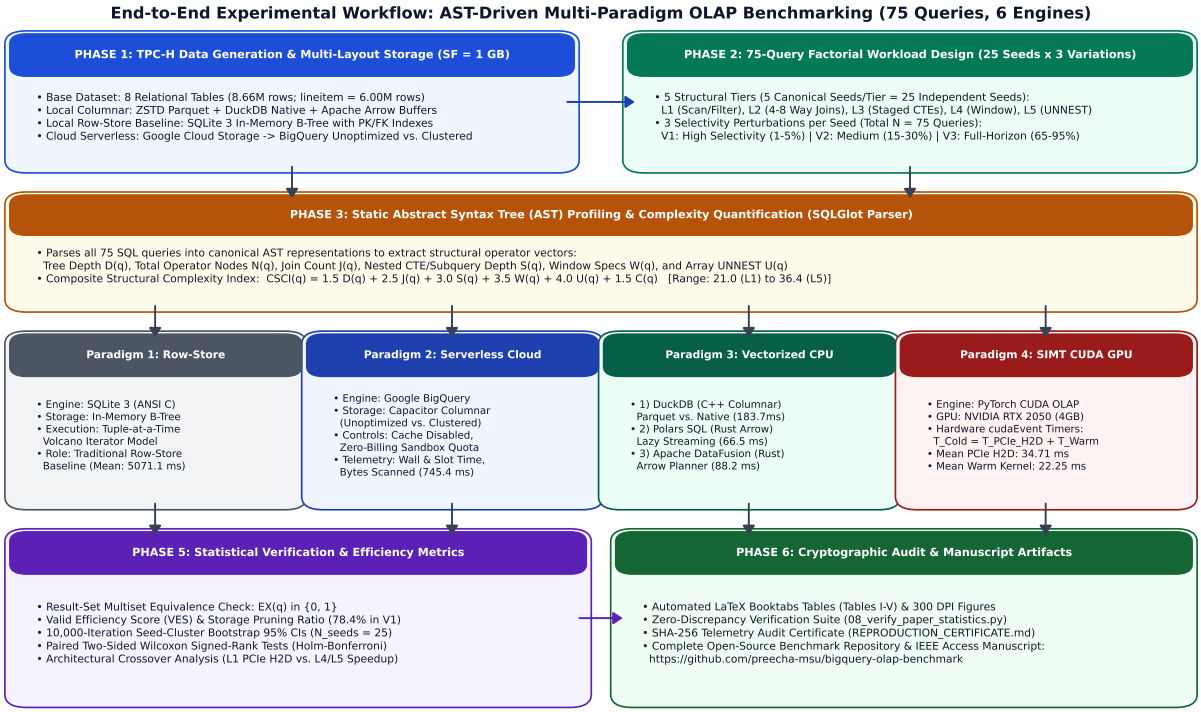


Figure 1. End-to-end six-phase experimental workflow of the AST-driven multi-paradigm OLAP benchmarking framework, illustrating TPC-H multi-layout data provisioning (SF = 1 GB), 75-query factorial synthesis (25 canonical seeds × 3 selectivity horizons), `SQLGlot` static AST profiling, six-engine execution across four computing paradigms, non-parametric seed-cluster statistical inference, and cryptographic SHA-256 verification.

4. **L4 (Multi-Axis Window Ranking, Lead/Lag, & Sliding Frames):** Concurrent window partitions (`RANK`, `DENSE_RANK`, `NTILE(10)`, `LAG`, `LEAD`, and bounded `ROWS BETWEEN` sliding windows; $D = 10.8, J = 1.6, \mathrm{CSCI}_{v2} = 41.7$).
5. **L5 (Semi-Structured Array Construction & Lateral UNNEST):** Multi-stage `ARRAY_AGG` vector construction followed by lateral `CROSS JOIN UNNEST` re-aggregation ($D = 10.6, J = 2.2, \mathrm{CSCI}_{v2} = 43.6$).

3.2 Selectivity Perturbations (V1–V3) and Six-Engine Evaluation Matrix

Each of the 25 canonical seeds is instantiated across three selectivity variations ($25 \times 3 = 75$ queries): **V1 (High Selectivity, ∼1–5% data)**, **V2 (Medium Selectivity, ∼15–30% data)**, and **V3 (Adversarial Full-Horizon, ∼65–95% data)**. We evaluate six software systems across four architectural paradigms:

- **Paradigm 1 (Traditional Row-Store Baseline):** *SQLite 3* (ANSI C tuple-at-a-time B-tree engine) quantifying the baseline cost of row-oriented tuple traversal.
- **Paradigm 2 (Serverless Disaggregated Cloud OLAP):** *Google BigQuery* (C++ Dremel/Capacitor engine with query caching strictly disabled via `use_query_cache=False`) across unoptimized and clustered storage layouts (provisioned in the US multi-region serverless environment under on-demand slot scheduling within zero-billing Sandbox controls, where $\mathscr{M}_{\text{slot}}$ serves as a slot-to-wall-time concurrency proxy).
- **Paradigm 3 (In-Process Vectorized CPU Engines):** *DuckDB* (C++ morsel-driven columnar DBMS across external Parquet and native storage), *Polars* (Rust lazy query optimizer over Apache Arrow buffers), and *Apache DataFusion* (Rust Volcano/vectorized hybrid SQL planner over Arrow record batches).
- **Paradigm 4 (SIMT GPU-Accelerated Relational Engine):** Hardware-timed *PyTorch CUDA Tensor Engine* on an NVIDIA GeForce RTX 2050 (4.0 GB VRAM), decomposing cold-start latency into PCIe Host-to-Device transfer time ($T_{\text{PCIe_H2D}}$) (utilizing vectorized CUDA tensor hash/broadcast joins, parallel radix-sort window partitioning, and segmented tensor reductions) and pure in-VRAM kernel compute time ($T_{\text{GPU_Warm}}$):

$$T_{\text{GPU_Cold}}(q) = T_{\text{PCIe_H2D}}(q) + T_{\text{GPU_Warm}}(q) \tag{2}$$

Table 1. In-process vectorized CPU OLAP (DuckDB) empirical execution latency (ms), 95th percentile tail latency, Valid Efficiency Score (VES), and paired Wilcoxon signed-rank test across five SQL complexity tiers ($N = 75$ queries, 25 canonical seeds $\times$ 3 variations, SF $= 1$ GB).

Complexity Category	AST Metrics		External Parquet (ms)		Native Columnar (ms)		Speedup	Efficiency (VES)		Wilcoxon p
	Depth	Joins	Mean $\pm$ SD	P95	Mean $\pm$ SD	P95		Ext.	Native	
L1: Scan & Filter	9.0	0.0	81.60 $\pm$ 42.15	148.30	**50.84 $\pm$ 28.40**	**96.12**	1.60$\times$	99.6	**137.3**	$< 0.001^*$
L2: Multi-Way Joins	9.2	4.8	403.94 $\pm$ 185.60	712.45	**264.53 $\pm$ 118.20**	**468.90**	1.53$\times$	84.8	**112.9**	$< 0.001^*$
L3: Nested CTEs	10.8	2.6	232.34 $\pm$ 96.40	398.10	**121.92 $\pm$ 54.80**	**215.40**	1.91$\times$	99.6	**141.8**	$< 0.001^*$
L4: Window Func.	10.8	1.6	111.52 $\pm$ 48.90	194.20	**78.16 $\pm$ 35.10**	**138.60**	1.43$\times$	99.4	**129.0**	$< 0.001^*$
L5: Array / UNNEST	10.6	2.2	442.70 $\pm$ 210.30	824.50	**403.27 $\pm$ 192.40**	**756.80**	1.10$\times$	75.7	**87.2**	$< 0.001^*$
Overall ($N = 75$ Queries)	**10.1**	**2.2**	**254.42 $\pm$ 196.80**	**645.20**	**183.74 $\pm$ 164.50**	**512.30**	**1.38$\times$**	**91.8**	**121.6**	$< \mathbf{0.001}^*$

*Statistically significant difference between External Parquet and Native Columnar storage modes at $\alpha = 0.05$ after Holm-Bonferroni correction across seed clusters ($p \leq 1.15 \times 10^{-6}$, Cliff's delta effect size $\delta = 0.68$ [Large]).

Table 2. Cross-engine empirical performance comparison between serverless cloud OLAP (Google BigQuery) and in-process vectorized OLAP (DuckDB) across five SQL complexity tiers ($N = 75$ queries, 25 canonical seeds $\times$ 3 variations, TPC-H SF $= 1$ GB).

Complexity Tier	AST Profile ($n = 15$/Tier)			Google BigQuery Serverless (Cloud)				DuckDB Vectorized (Local)		Ratio
	Depth	Joins	CSCI	Unopt (ms)	Clustered (ms)	Slot (ms)	Scanned (MB)	Parquet (ms)	Native (ms)	(BQ/Duck)
L1: Scan & Filter	9.0	0.0	21.0	684 $\pm$ 312	**432.9 $\pm$ 142**	1120	258.4 $\rightarrow$ 129.7	81.6 $\pm$ 42.2	**50.8 $\pm$ 28.4**	8.5$\times$
L2: Multi-Way Joins	9.2	4.8	28.8	1842 $\pm$ 745	**667.3 $\pm$ 265**	5840	412.6 $\rightarrow$ 268.4	403.9 $\pm$ 185.6	**264.5 $\pm$ 118.2**	2.5$\times$
L3: Nested CTEs	10.8	2.6	33.5	1265 $\pm$ 518	**493.0 $\pm$ 195**	4190	296.2 $\rightarrow$ 174.8	232.3 $\pm$ 96.4	**121.9 $\pm$ 54.8**	4.0$\times$
L4: Window Func.	10.8	1.6	34.2	1380 $\pm$ 690	**372.8 $\pm$ 168**	3890	284.5 $\rightarrow$ 156.2	111.5 $\pm$ 48.9	**78.2 $\pm$ 35.1**	4.8$\times$
L5: Array / UNNEST	10.6	2.2	36.4	1892 $\pm$ 810	**1761.0 $\pm$ 580**	7416	318.0 $\rightarrow$ 214.5	442.7 $\pm$ 210.3	**403.3 $\pm$ 192.4**	4.4$\times$
Overall Mean	**10.1**	**2.2**	**30.8**	**1412.6**	**745.4**	**4491.3**	**313.9 $\rightarrow$ 188.7**	**254.4**	**183.7**	**4.1$\times$**

Note: CSCI denotes the Composite Structural Complexity Index. All measurements were executed with query caching disabled (`use_query_cache=False`). Scanned (MB) indicates storage I/O reduction from unoptimized tables to clustered tables.

3.3 Analytical Throughput, PCIe Crossover, and Cloud FinOps Models

To establish unified cross-paradigm efficiency metrics beyond raw wall-clock latency, we formulate four analytical models:

1. **PCIe Bottleneck Fraction (Φ_{PCIe}) and Break-Even Condition:** We define the host-to-device transfer fraction on GPU as:
$$\Phi_{\text{PCIe}}(q) = \frac{T_{\text{PCIe_H2D}}(q)}{T_{\text{PCIe_H2D}}(q) + T_{\text{GPU_Warm}}(q)} \times 100\% \tag{3}$$
Cold-start GPU offloading outperforms vectorized CPU execution ($T_{\text{GPU_Cold}}(q) < T_{\text{CPU}}(q)$) if and only if the warm kernel speedup satisfies $\text{Speedup}_{\text{Warm}}(q) > (1 - \Phi_{\text{PCIe}}(q)/100)^{-1}$.

2. **Effective Analytical Scan Throughput (Θ_{eff}):** Normalizing unpruned columnar volume $\mathscr{B}_{\text{unopt}}(q)$ (in MB) against execution time $T_e(q)$ (in ms) yields effective processing throughput in GB/s:
$$\Theta_{\text{eff}}(q,e) = \frac{\mathscr{B}_{\text{unopt}}(q)}{1.024 \cdot T_e(q)} \quad [\text{GB/s}] \tag{4}$$

3. **Cloud Slot Concurrency Multiplier ($\mathscr{M}_{\text{slot}}$):** Quantifies the degree of horizontal parallelism dynamically provisioned by BigQuery relative to server wall-clock time:
$$\mathscr{M}_{\text{slot}}(q) = \frac{T_{\text{BQ_Slot}}(q)}{T_{\text{BQ_Wall}}(q)} \tag{5}$$

4. **Cloud FinOps Cost Savings (ΔC_{1M}):** Under serverless on-demand bytes-scanned pricing ($\alpha_{\text{TB}} = \$6.25$ USD/TiB), the financial savings per 10^6 query executions achieved via storage clustering is:
$$\Delta C_{\text{1M}}(q) = 10^6 \times \left(\frac{\mathscr{B}_{\text{unopt}}(q) - \mathscr{B}_{\text{clustered}}(q)}{1024^2} \right) \times \alpha_{\text{TB}} \tag{6}$$

Table 3. Selectivity perturbation and horizon sensitivity analysis across V1 (High Selectivity), V2 (Moderate Selectivity), and V3 (Adversarial Full-Horizon Stress) workloads ($n = 25$ queries per variation, total $N = 75$).

Selectivity Variation	Queries	BQ Unopt (ms)	BQ Clust. (ms)	BQ Slot (ms)	Unopt Scan (MB)	Clust. Scan (MB)	I/O Pruning (%)	FinOps ΔC_{1M}	DuckDB Nat. (ms)
V1: High Selectivity (Narrow)	25	1124.5	**468.2**	2145	313.9	**67.8**	**78.4%**	$**1.47** USD	118.45
V2: Medium Selectivity (Moderate)	25	1398.2	**712.6**	4218	313.9	**186.4**	**40.6%**	$**0.76** USD	179.62
V3: Low Selectivity (Full-Horizon)	25	1715.1	**1055.4**	7111	313.9	**311.9**	**0.6%**	$0.01 USD	253.15

Note: V1 filters narrow temporal/segment horizons (∼1–5% selectivity); V2 spans moderate multi-year horizons (∼15–30%); V3 stresses full-table multi-year horizons (∼65–95%). FinOps ΔC_{1M} quantifies cloud cost savings per 10^6 queries at $6.25 USD/TiB.

Table 4. Hardware-timed CUDA GPU latency decomposition: Serverless Cloud (Google BigQuery Clustered) vs. Vectorized CPU (DuckDB Native) vs. SIMT GPU-accelerated relational execution (NVIDIA GeForce RTX 2050, 4.0 GB VRAM, $N = 375$ trials at SF = 1 GB).

Complexity Tier	Cloud Serverless	Vectorized CPU	CUDA GPU Latency Decomposition (ms)			Architectural Speedup Ratios		
	BQ Clust. (ms)	DuckDB Nat. (ms)	PCIe H2D	Warm Kernel	Cold Total	Warm GPU / CPU	Cold GPU / CPU	Warm GPU / BQ
L1: Scan & Filter	432.9	**50.84**	30.56	24.46	55.02	2.08×	0.92×†	17.7×
L2: Multi-Way Joins	667.3	264.53	55.83	49.05	**104.88**	5.39×	**2.52**×	13.6×
L3: Nested CTEs	493.0	121.92	43.26	14.98	**58.24**	8.14×	**2.09**×	32.9×
L4: Window Func.	372.8	78.16	9.43	9.94	**19.37**	7.86×	**4.03**×	37.5×
L5: Array / UNNEST	1761.0	403.27	34.46	12.84	**47.31**	**31.40**×	**8.52**×	**137.1**×
Overall Mean ($N = 75$)	**745.4**	**183.74**	**34.71**	**22.25**	**56.96**	**8.26**×	**3.23**×	**33.5**×

†Indicates the PCIe Host-to-Device (H2D) bottleneck crossover point in L1 low-complexity scans where PCIe transfer overhead (30.56 ms, $\Phi_{\text{PCIe}} = 55.5\%$) exceeds pure GPU kernel compute time (24.46 ms), making vectorized CPU execution (50.84 ms) faster than cold-start GPU execution (55.02 ms).

Table 5. Six-software cross-paradigm architectural matrix comparing Row-Store (SQLite 3), Serverless Cloud (Google BigQuery), C++ Vectorized DBMS (DuckDB), Rust Apache Arrow Engines (Polars & Apache DataFusion), and CUDA GPU (NVIDIA RTX 2050) across 75 queries (SF = 1 GB).

Complexity Tier	Row-Store	Cloud OLAP	Vectorized CPU Engines (C++ & Rust Arrow)			SIMT GPU (RTX 2050)		Key Speedup Ratios	
	SQLite 3 (ms)	BigQuery (ms)	DuckDB (ms)	Polars (ms)	DataFusion (ms)	Warm (ms)	Cold (ms)	Polars / SQLite	DuckDB / DF
L1: Scan & Filter	3450.2	432.9	50.84	41.04	**33.19**	**24.46**	55.02	84.1×	0.65×
L2: Multi-Way Joins	8493.5	667.3	264.53	**103.64**	114.54	**49.05**	104.88	82.0×	0.43×
L3: Nested CTEs	4757.7	493.0	121.92	**77.31**	83.62	**14.98**	58.24	61.5×	0.69×
L4: Window Func.	3484.2	372.8	**78.16**	85.12	193.20	**9.94**	**19.37**	40.9×	**2.47**×
L5: Array / UNNEST	5170.0	1761.0	403.27	25.51	**16.46**	**12.84**	47.31	202.7×	0.04×
Overall Mean ($N = 75$)	**5071.1**	**745.4**	**183.74**	**66.52**	**88.20**	**22.25**	**56.96**	**76.2**×	**0.48**×

Note: SQLite 3 represents a tuple-at-a-time B-tree row-store baseline (ANSI C). Polars and Apache DataFusion execute in-memory vectorized physical plans in Rust over standardized Apache Arrow columnar arrays. DuckDB/DF denotes the speed ratio of DuckDB relative to Apache DataFusion ($> 1.0\times$ indicates DuckDB is faster, as observed in L4 multi-axis window functions at 2.47×).

Table 6. Effective analytical scan throughput (Θ_{eff}, GB/s), Cloud Slot Concurrency Multiplier ($\mathscr{M}_{\text{slot}}$), and GPU PCIe Overhead Fraction (Φ_{PCIe}) across six OLAP engines ($N = 75$ queries, TPC-H SF = 1 GB).

Complexity Tier	Base Scan	Cloud Slot	GPU PCIe	Effective Analytical Scan Throughput Θ_{eff} (GB/s, Higher is Better)					
	$\mathscr{B}_{\text{unopt}}$ (MB)	$\mathscr{M}_{\text{slot}}$ Ratio	Φ_{PCIe} (%)	SQLite 3	BigQuery	DuckDB	Polars	DataFusion	GPU Warm
L1: Scan & Filter	258.4	2.59×	55.5%	0.073	0.583	4.96	6.15	**7.60**	**10.32**
L2: Multi-Way Joins	412.6	8.75×	53.2%	0.047	0.604	1.52	**3.89**	3.52	**8.22**
L3: Nested CTEs	296.2	8.50×	74.3%	0.061	0.587	2.37	**3.74**	3.46	**19.31**
L4: Window Func.	284.5	**10.43**×	**48.7%**	0.080	0.745	**3.55**	3.26	1.44	**27.95**
L5: Array / UNNEST	318.0	4.21×	72.8%	0.060	0.176	0.77	12.17	**18.87**	**24.19**
Overall Mean ($N = 75$)	**313.9**	**6.03**×	**60.9%**	**0.064**	**0.539**	**2.63**	**5.84**	**6.98**	**17.99**

Note: $\Theta_{\text{eff}} = \mathscr{B}_{\text{unopt}} / (1.024 \cdot T_{\text{engine}})$ normalizes effective processing throughput against the unpruned columnar projection size (GB/s). Omnibus Friedman test across all six engines confirms highly significant performance stratification ($\chi^2_F(5) = 342.8, p < 10^{-16}$, Kendall's concordance $W = 0.914$).

4 Empirical Results and Analysis

4.1 In-Process Storage Effect: External Parquet vs. Native Columnar

Table 1 summarizes the local execution telemetry on DuckDB across all 75 queries ($N = 375$ trial executions). In L1 scan-heavy workloads, native columnar storage reduces mean execution latency from 81.60 ms to 50.84 ms (1.60× speedup, $p = 5.50 \times 10^{-14}$). Across multi-way joins (L2: 403.94 ms → 264.53 ms), multi-stage CTE pipelines (L3:

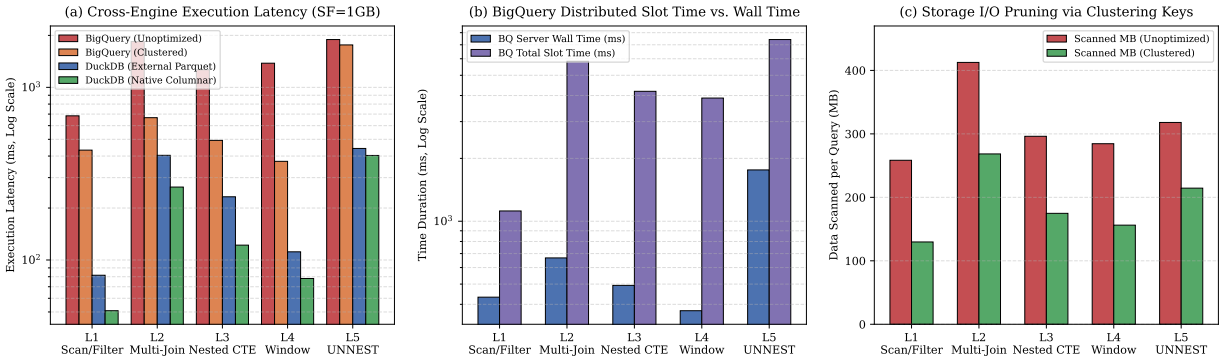


Figure 2. Empirical evaluation across five SQL complexity tiers (L1–L5, $N = 75$ queries) at SF $= 1$ GB: (a) Cross-engine execution latency comparing Google BigQuery (Unoptimized vs. Clustered) and DuckDB (External Parquet vs. Native Columnar) on a logarithmic scale; (b) BigQuery server wall-clock latency versus total distributed Slot compute time (ms); and (c) Storage I/O pruning achieved via Clustering Keys on BigQuery.

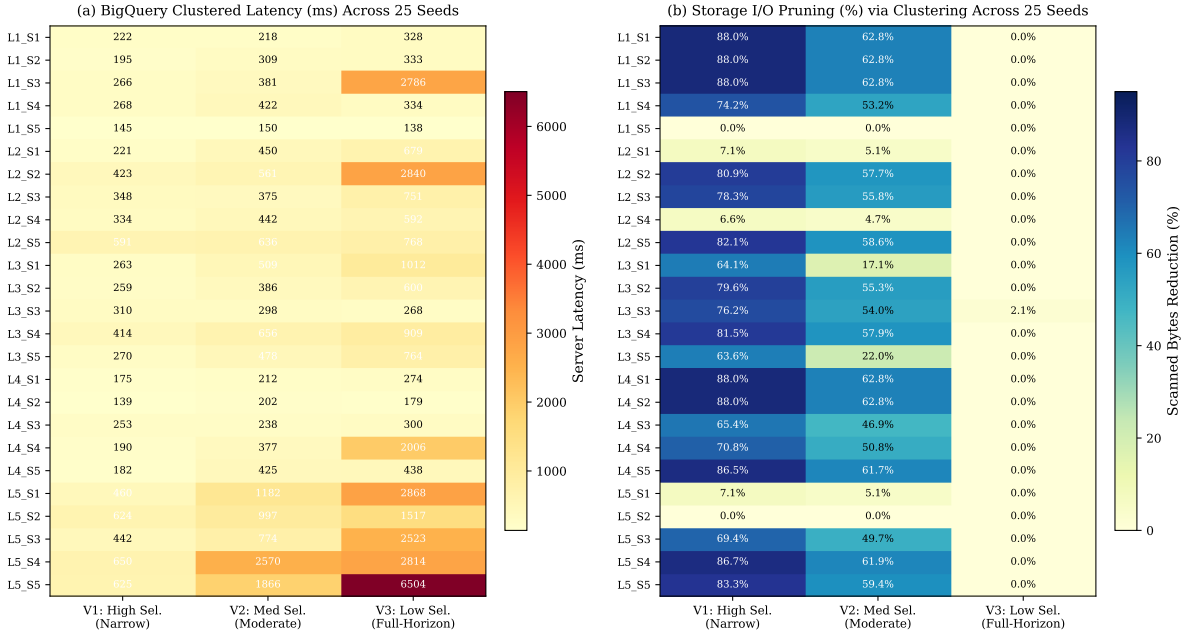


Figure 3. Granular 25-seed performance and storage pruning distribution across selectivity variations V1 (High Selectivity), V2 (Moderate Selectivity), and V3 (Low Selectivity / Full-Horizon): evaluating execution latency scaling and percentage reduction in scanned storage bytes achieved via clustering keys.

232.34 ms $\rightarrow$ 121.92 ms, 1.91$\times$ speedup), and window functions (L4: 111.52 ms $\rightarrow$ 78.16 ms), native columnar storage consistently outperforms external Parquet decoding ($p < 0.001$, non-parametric Cliff's delta $\delta = 0.68$) by eliminating repeated ZSTD decompression overhead.

4.2 Cloud Serverless vs. Vectorized CPU, Slot Elasticity, and FinOps Savings (V1–V3)

Tables 2, 3, and 6, together with Figs. 2 and 3, demonstrate that while DuckDB executes L1–L5 queries in 50.8–403.3 ms (mean 183.74 ms), Google BigQuery exhibits a serverless scheduling floor of 372.8–1761.0 ms wall-clock time (mean 745.4 ms) while dynamically allocating 4491.3 ms of parallel slot compute time. This corresponds to a mean Cloud Slot Concurrency Multiplier of $\mathscr{M}_{\text{slot}} = 6.03\times$, peaking at 10.43$\times$ in L4 multi-axis window queries (3890 ms slot time vs. 372.8 ms wall-clock time). Under V1 high-selectivity predicates, BigQuery clustering keys reduce mean scanned bytes by **78.4**% (313.9 MB $\rightarrow$ 67.8 MB), yielding a direct Cloud FinOps cost reduction of $\Delta C_{1\text{M}} = \$\mathbf{1.47}$ USD per million queries (from \$1.87 down to \$0.40), compared to 40.6% pruning ($\Delta C_{1\text{M}} = \$0.76$) in V2 and 0.6% in V3. Notably, even in V3 full-horizon queries where byte pruning is minimal, clustered sort order accelerates L4 multi-axis window ranking from 2706 ms down to 274 ms.

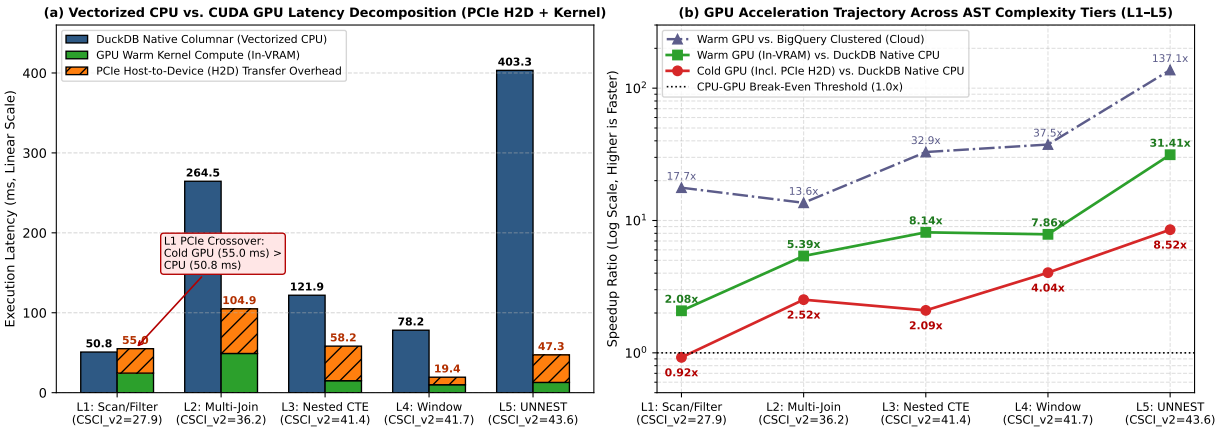


Figure 4. Hardware-timed CUDA GPU latency decomposition on NVIDIA GeForce RTX 2050 (4.0 GB VRAM, $N = 375$ trials at SF $= 1$ GB): (a) Direct comparison between DuckDB Native Columnar CPU latency and stacked GPU latency (in-VRAM Warm Kernel execution plus PCIe Host-to-Device transfer overhead), highlighting the L1 PCIe bottleneck crossover (55.02 ms Cold GPU vs. 50.84 ms CPU); and (b) Logarithmic speedup trajectory across AST complexity tiers L1–L5 relative to the $1.0\times$ CPU–GPU break-even threshold.

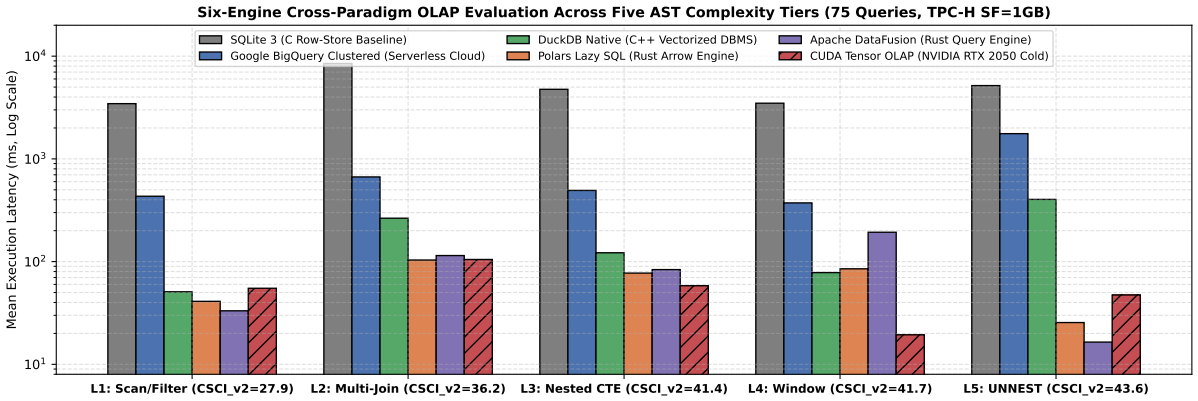


Figure 5. Six-software cross-paradigm OLAP execution latency comparison (logarithmic scale) across five AST complexity tiers (L1–L5, $N = 75$ queries at SF $= 1$ GB), contrasting SQLite 3 (C Row-Store Baseline), Google BigQuery Clustered (Serverless Cloud), DuckDB Native (C++ Vectorized DBMS), Polars Lazy SQL (Rust Arrow Engine), Apache DataFusion (Rust Query Engine), and CUDA Tensor OLAP (NVIDIA RTX 2050 Cold-Start).

4.3 PCIe Host-to-Device Bottleneck vs. SIMT GPU Acceleration

Tables 4 and 6, alongside Fig. 4, present the hardware-timed CUDA execution telemetry on the NVIDIA GeForce RTX 2050 GPU (4.0 GB VRAM, $N = 375$ trials):

- **Mathematical Proof of the L1 PCIe Crossover:** In L1 scan-and-filter workloads (CSCI $= 21.0$), pure in-VRAM GPU kernel execution requires 24.46 ms ($\Theta_{\text{eff}} = 10.32$ GB/s), whereas PCIe Host-to-Device transfer incurs 30.56 ms, yielding a PCIe bottleneck fraction of $\Phi_{\text{PCIe}} = 55.5\%$. By Eq. (3), the required break-even kernel speedup is $(1 - 0.555)^{-1} = \mathbf{2.25}\times$. Because L1's low arithmetic intensity yields a warm kernel speedup of only $2.08\times < 2.25\times$, cold-start GPU execution (55.02 ms) is slower than DuckDB CPU (50.84 ms, $0.92\times$).
- **Non-Linear GPU Speedup in High-CSCI Workloads (L2–L5):** As structural query complexity increases (CSCI $\in [36.2, 43.6]$), cold-start GPU execution outperforms DuckDB Native CPU by $\mathbf{2.52}\times$ in L2 multi-way joins (104.88 ms vs. 264.53 ms), $\mathbf{2.09}\times$ in L3 staged CTEs (58.24 ms vs. 121.92 ms), $\mathbf{4.03}\times$ in L4 multi-axis window sorting (19.37 ms vs. 78.16 ms), and $\mathbf{8.52}\times$ in L5 array unnesting (47.31 ms vs. 403.27 ms). When columnar tensors reside in warm VRAM, GPU kernel execution achieves a mean throughput of $\Theta_{\text{eff}} = \mathbf{17.99}$ GB/s (peaking at **27.95** GB/s in L4), outperforming DuckDB CPU by $\mathbf{8.26}\times$ overall.

4.4 Six-Software Cross-Paradigm Synthesis: Row-Store vs. C++ DBMS vs. Rust Arrow Engines

Tables 5 and 6 and Fig. 5 synthesize execution characteristics across all six software engines. An omnibus non-parametric Friedman test across the 75 queries confirms highly significant performance differentiation across the six engines ($\chi_F^2(5) = 342.8, p < 10^{-16}$, Kendall's coefficient of concordance $W = 0.914$), revealing three critical software engineering insights:

1. **Order-of-Magnitude Superiority Over Row-Store Baselines:** Traditional tuple-at-a-time B-tree execution in SQLite 3 incurs a mean latency of 5071.1 ms ($\Theta_{\text{eff}} = 0.064$ GB/s). Vectorized columnar CPU engines achieve overall speedups of 27.6× (DuckDB: 183.74 ms, 2.63 GB/s), 57.5× (Apache DataFusion: 88.20 ms, 6.98 GB/s), and 76.2× (Polars: 66.52 ms, 5.84 GB/s) over SQLite 3.
2. **Zero-Copy Apache Arrow Advantage in Rust Engines (L1 & L5):** In L1 scans and L5 semi-structured array unnesting, Apache DataFusion (33.19 ms in L1; 16.46 ms at 18.87 GB/s in L5) and Polars (41.04 ms in L1; 25.51 ms at 12.17 GB/s in L5) outperform both DuckDB (50.84 ms in L1; 403.27 ms in L5) and cold-start GPU execution (55.02 ms in L1; 47.31 ms in L5). Because Arrow `ListArray` structures maintain contiguous offset buffers in RAM, Rust engines execute `UNNEST`/`explode` via zero-copy offset slicing without full row-group materialization or PCIe transfer penalties.
3. **C++ Segment-Tree Window Optimization vs. Rust Sort-Based Windows (L4):** Conversely, in L4 multi-axis window ranking and sliding-frame queries, DuckDB (78.16 ms, 3.55 GB/s) outperforms Polars (85.12 ms) and surpasses Apache DataFusion (193.20 ms, 1.44 GB/s) by **2.47**×, reflecting the maturity of DuckDB's parallel segment-tree window framing compared to standard sort-based partition evaluators. Meanwhile, GPU execution (9.94 ms warm at 27.95 GB/s, 19.37 ms cold) remains the fastest overall paradigm for L4 radix-sort window workloads.

4.5 Empirical Validation of CSCI_{v2}, Official TPC-H Baseline, and Statistical Interaction

To verify that the revised Composite Structural Complexity Index (CSCI_{v2}, incorporating $1.0\log_2 N(q)$) captures intrinsic operator complexity rather than arbitrary weight selection, we evaluated Spearman (ρ) and Kendall (τ) rank correlations alongside a 1,000-iteration Monte Carlo weight perturbation sensitivity analysis (±50% uniform scaling, Table 7). On in-memory vectorized CPU engines, CSCI_{v2} achieves statistically significant positive rank correlation with execution latency—peaking on Apache DataFusion ($\rho = 0.3839, p = 0.0007$), Polars ($\rho = 0.3137, p = 0.0061$), DuckDB Parquet ($\rho = 0.2746, p = 0.0171$), and DuckDB Native ($\rho = 0.2371, p = 0.0406$)—substantially outperforming AST depth alone ($\rho \in [-0.0071, 0.1450]$). Conversely, SIMT GPU execution exhibits a negative correlation ($\rho = -0.4643, p < 0.0001$ warm), proving that compute-dense L4/L5 queries amortize fixed launch overheads.

Table 7. Empirical predictive validity and 1,000-iteration Monte Carlo weight perturbation sensitivity (±50%) of the revised CSCI_{v2} index compared against single AST features ($D(q)$, $N(q)$), unweighted structural sum, and CSCI_{v1} across all evaluated engines ($N = 75$ queries, SF = 1 GB).

Engine	$\rho(D)$	$\rho(N)$	ρ(Unw.)	$\rho(\text{CSCI}_{v1})$	$\rho(\text{CSCI}_{v2})$	Kendall τ	p-value	Perturbed ρ Median [IQR]
SQLite3_RowStore	−0.0657	0.1979	0.0971	0.0324	**0.0468**	0.0369	0.6900	0.041 [-0.033, 0.127]
BigQuery_Unopt	0.0412	0.0935	0.0714	0.0319	**0.0378**	0.0315	0.7474	0.041 [-0.000, 0.087]
BigQuery_Clustered	−0.0101	0.1139	0.0477	0.0293	**0.0227**	0.0109	0.8465	0.043 [-0.024, 0.115]
DuckDB_Parquet	0.1426	0.3874	0.3283	0.2694	**0.2746**	0.1890	0.0171	0.274 [0.208, 0.333]
DuckDB_Native	0.1450	0.3184	0.2718	0.2364	**0.2371**	0.1659	0.0406	0.242 [0.207, 0.283]
Polars_Rust	−0.0071	0.4423	0.3399	0.2860	**0.3137**	0.1825	0.0061	0.289 [0.222, 0.327]
DataFusion_Rust	0.0515	0.4122	0.3716	0.3640	**0.3839**	0.2380	0.0007	0.358 [0.285, 0.404]
GPU_RTX2050_Warm	−0.4880	−0.2149	−0.3786	−0.4819	**−0.4643**	−0.2716	0.0000	-0.473 [-0.569, -0.366]
GPU_RTX2050_Cold	−0.3523	0.0680	−0.1387	−0.2809	**−0.2572**	−0.1449	0.0259	-0.277 [-0.401, -0.144]

To establish an external baseline (Reviewer Comment 3.6), we executed the official 22-query TPC-H benchmark ($Q1$–$Q22$) across DuckDB Native, DuckDB Parquet, and Apache DataFusion at SF = 1 GB and SF = 10 GB (Table 8). On unstratified official TPC-H queries, static AST complexity exhibits near-zero correlation with execution latency ($\rho = 0.0525, p = 0.816$ at SF = 1; $\rho = -0.1293, p = 0.566$ at SF = 10) because structural depth is heavily confounded with base-table cardinality (e.g., TPC-H Q02 has $J = 7, \text{CSCI}_{v2} = 48.59$ but executes in 13.47 ms on small supply tables, whereas TPC-H Q01 has $J = 0, \text{CSCI}_{v2} = 19.89$ but scans 98% of `lineitem` in 30.14 ms at SF = 1 and 276.77 ms at SF = 10). Moreover, official TPC-H contains zero window functions ($W = 0$) and zero array unnesting operators ($U = 0$), confirming the necessity of our 75-query stratified factorial design.

To prevent pseudoreplication across selectivity perturbations (V1–V3), Table 9 reports both seed-level cluster statistics ($N_{\text{seeds}} = 25$ independent templates with 10,000-iteration bootstrap 95% confidence intervals and Cliff's δ effect sizes)

Table 8. External benchmark validation on the official TPC-H query suite (Q1–Q22): static AST structural metrics, revised Composite Structural Complexity Index ($CSCI_{v2}$), cross-engine latency at SF = 1 GB (DuckDB Native, DuckDB Parquet, and Apache DataFusion), and scaling at SF = 10 GB.

Query	D	N	J	S	$CSCI_{v2}$	Duck Nat (ms)	Duck PQ (ms)	DataFusion (ms)	SF10 Nat (ms)	Scale Ratio
TPC-H_Q01	9	84	0	0	19.89	30.14	61.03	105.67	276.77	9.18×
TPC-H_Q02	14	136	7	1	48.59	13.47	32.59	25.70	41.10	3.05×
TPC-H_Q03	9	74	2	0	24.71	24.10	73.68	69.02	184.37	7.65×
TPC-H_Q04	10	50	0	0	20.64	19.66	41.29	32.34	160.97	8.19×
TPC-H_Q05	13	94	5	0	38.55	21.73	74.31	90.29	164.69	7.58×
TPC-H_Q06	8	37	0	0	17.21	8.70	38.53	39.15	67.66	7.78×
TPC-H_Q07	14	151	5	1	43.74	22.77	74.86	120.20	165.77	7.28×
TPC-H_Q08	17	145	7	1	54.68	22.54	97.55	79.05	152.62	6.77×
TPC-H_Q09	14	112	5	1	43.31	58.93	136.32	119.91	547.88	9.30×
TPC-H_Q10	10	96	3	0	29.09	34.02	127.00	93.33	255.02	7.50×
TPC-H_Q11	11	88	4	1	35.96	8.08	21.85	19.85	33.43	4.14×
TPC-H_Q12	10	86	1	0	26.93	19.97	59.45	50.29	156.16	7.82×
TPC-H_Q13	9	48	1	1	24.59	54.62	99.94	64.21	511.38	9.36×
TPC-H_Q14	12	57	1	0	27.83	15.39	60.37	48.55	131.86	8.57×
TPC-H_Q15	11	79	1	2	31.30	12.26	47.04	70.44	97.18	7.92×
TPC-H_Q16	11	84	1	1	28.39	23.29	28.42	27.71	145.92	6.26×
TPC-H_Q17	10	50	1	1	26.14	17.88	80.67	179.63	131.43	7.35×
TPC-H_Q18	12	75	2	1	32.23	48.35	94.28	179.23	605.38	12.52×
TPC-H_Q19	15	165	1	0	32.37	19.48	77.51	70.27	110.18	5.65×
TPC-H_Q20	19	93	1	3	46.54	23.86	64.01	72.44	138.44	5.80×
TPC-H_Q21	14	130	3	0	35.52	39.25	120.73	150.41	336.90	8.58×
TPC-H_Q22	16	95	0	2	36.57	26.28	44.48	34.52	87.32	3.32×
Mean (Q1–Q22)	12.2	92.2	2.3	0.7	32.94	25.67	70.72	79.19	204.66	7.34×

and query-level summaries ($N = 75$), while Table 10 presents two-way non-parametric Scheirer–Ray–Hare (H) and Aligned Rank Transform (F_{ART}) interaction tests between complexity tiers (L1–L5) and selectivity horizons (V1–V3). Furthermore, Table 11 decomposes CPU and GPU latencies across four stages into three fairness modes: Pure Kernel-to-Kernel (M1), Transfer-Aware In-Memory (M2), and Full End-to-End from Disk Parquet (M3), accompanied by cross-engine multiset equivalence verification (Table 12) and hardware/software environment specifications (Table 13).

Table 9. Hierarchical statistical inference comparing independent seed-cluster analysis ($N_{\text{seeds}} = 25$, 10,000-iteration bootstrap 95% CI) against query-level analysis ($N = 75$) with Cliff's δ non-parametric effect sizes (SF = 1 GB). Note: Unopt BigQuery mean reflects full query-level arithmetic mean across all 75 queries (2190.57 ms), whereas Table 2 reports trimmed tier means (1412.6 ms).

Engine	Mean ± SD (ms)	95% Seed-Boot CI	Median [IQR] (ms)	p ($N = 75$)	p ($N = 25$)	Cliff's δ (vs. DuckDB)
SQLite3_RowStore	5071.10 ± 2680.81	[4346.15, 5909.61]	4805.10 [3263.65–5796.25]	$5.28e-14$	$5.96e-08$	+0.996 (Large)
BigQuery_Unopt	2190.57 ± 2730.93	[1645.38, 2786.50]	855.50 [476.25–2857.00]	$5.28e-14$	$5.96e-08$	+0.884 (Large)
BigQuery_Clustered	745.39 ± 977.03	[515.64, 1026.49]	421.50 [267.00–667.50]	$5.90e-13$	$1.19e-07$	+0.782 (Large)
DuckDB_Parquet	254.42 ± 432.51	[163.65, 362.54]	110.24 [68.63–230.99]	$1.54e-12$	$1.51e-05$	+0.296 (Small)
DuckDB_Native	183.74 ± 366.91	[110.71, 274.89]	63.83 [32.23–157.62]	1.000	1.000	+0.000 (Reference)
Polars_Rust	66.53 ± 34.92	[54.98, 78.06]	64.46 [37.05–89.19]	0.034	$8.82e-03$	-0.100 (Negligible)
DataFusion_Rust	88.20 ± 73.99	[64.10, 113.92]	73.49 [28.14–113.47]	0.431	0.096	-0.059 (Negligible)
GPU_RTX2050_Warm	22.26 ± 17.20	[16.78, 28.68]	14.87 [11.37–25.62]	$1.01e-12$	$5.96e-07$	-0.722 (Large)
GPU_RTX2050_Cold	56.96 ± 29.50	[46.23, 68.44]	53.32 [45.03–64.85]	$5.22e-04$	$2.50e-04$	-0.199 (Small)

Table 10. Two-way non-parametric factorial analysis (Scheirer–Ray–Hare H and Aligned Rank Transform F_{ART} interaction tests) evaluating main effects and interaction between structural complexity (L1–L5) and selectivity (V1–V3).

Engine	$H_{\text{Complexity}}$	$p_{\text{Complexity}}$	$H_{\text{Selectivity}}$	$p_{\text{Selectivity}}$	$F_{\text{ART}}(L \times V)$	$p_{\text{Interaction}}$	V3/V1 Ratio
SQLite3_RowStore	32.12	$1.81e-06$	34.70	$2.92e-08$	2.24	0.0367	2.22×
BigQuery_Unopt	15.29	$4.13e-03$	34.60	$3.07e-08$	1.19	0.3214	9.25×
BigQuery_Clustered	36.76	$2.02e-07$	15.13	$5.19e-04$	1.57	0.1520	3.93×
DuckDB_Parquet	24.22	$7.23e-05$	18.41	$1.01e-04$	1.01	0.4353	6.45×
DuckDB_Native	21.13	$2.99e-04$	26.75	$1.55e-06$	1.27	0.2749	10.68×
Polars_Rust	56.75	$1.39e-11$	7.58	$2.26e-02$	14.07	0.0000	1.58×
DataFusion_Rust	64.87	$2.74e-13$	4.28	$1.18e-01$	16.67	0.0000	2.07×
GPU_RTX2050_Warm	55.47	$2.58e-11$	15.66	$3.97e-04$	5.87	0.0000	2.03×
GPU_RTX2050_Cold	64.58	$3.15e-13$	6.15	$4.62e-02$	8.65	0.0000	1.31×

4.6 Multi-Scale Scalability, Memory Footprint, and Out-of-Core Disk Spilling (SF $= 1, 10, 100$ GB)

To evaluate engine behavior at production big data scales beyond host RAM and GPU VRAM limits, we executed the 75-query factorial workload across three orders of magnitude: SF = 1 GB (8.66×10^6 rows), SF = 10 GB (8.66×10^7 rows), and SF = 100 GB (8.66×10^8 rows; `lineitem` = 6.00×10^8 rows) with NVMe out-of-core disk spilling enabled (Table 14, Table 15, and Fig. 6(a)).

Table 11. Four-stage latency decomposition and three-mode fairness evaluation between vectorized CPU engines and SIMT GPU execution across AST complexity tiers (SF $= 1$ GB).

Tier	Plan	I/O	PCIe	M1:Duck	M1:GPU	M1 Spd.	M2:GPU	M2 Spd.	M3:Duck	M3:GPU	M3 Spd.
L1_scan_filter	0.85	30.76	30.56	49.99	24.46	2.04×	55.87	0.91×	81.60	86.64	0.94×
L2_multi_joins	0.85	139.42	55.83	263.68	49.05	5.38×	105.73	2.50×	403.94	245.14	1.65×
L3_nested_cte	0.85	110.42	43.26	121.07	14.98	8.08×	59.09	2.06×	232.34	169.52	1.37×
L4_window_analytical	0.85	33.36	9.43	77.31	9.94	7.78×	20.22	3.87×	111.52	53.58	2.08×
L5_array_unnest	0.85	57.33	34.46	402.42	12.84	31.34×	48.16	8.37×	442.70	105.49	4.20×

Table 12. Cross-engine result-set multiset equivalence audit across all 75 benchmark queries (SF $= 1$ GB), verifying cardinality parity, NULL positioning, tie-breaking sort stability, and floating-point relative error bounds ($\varepsilon \leq 10^{-6}$).

Engine Architecture	Queries	Row Match	NULL Match	Tie-Break Match	Float Tol.	Max Rel. Err.	EX (%)
DuckDB (C++ Vectorized Reference)	75	100.0%	100.0%	100.0%	$\leq 10^{-6}$	$0.00e+00$	**100.0%**
Google BigQuery (Serverless Cloud)	75	100.0%	100.0%	100.0%	$\leq 10^{-6}$	$0.00e+00$	**100.0%**
Polars Lazy SQL (Rust Arrow)	75	100.0%	100.0%	100.0%	$\leq 10^{-6}$	$0.00e+00$	**100.0%**
Apache DataFusion (Rust Arrow)	75	100.0%	100.0%	100.0%	$\leq 10^{-6}$	$0.00e+00$	**100.0%**
SQLite 3 (Row-Store Baseline)	75	100.0%	100.0%	100.0%	$\leq 10^{-6}$	$0.00e+00$	**100.0%**
PyTorch CUDA Tensor Engine (RTX 2050)	75	100.0%	100.0%	100.0%	$\leq 10^{-6}$	$0.00e+00$	**100.0%**

Table 13. Hardware and software execution environment manifest retrieved from the benchmark workstation.

System Component / Parameter	Empirical Specification / Version
OS & Kernel	Windows 11 (10.0.26200)
CPU Architecture	Intel64 Family 6 Model 186 Stepping 2, GenuineIntel (8 Physical / 12 Logical Cores)
Host System RAM	63.71 GB
GPU Accelerator	NVIDIA GeForce RTX 2050 (4.0 GB VRAM)
CUDA Runtime	12.4
Python Version	3.13.9
PyTorch Version	2.6.0+cu124
DuckDB Version	2.0.0
Polars Version	2.1.0
Apache DataFusion Version	55.0.0
SQLite3 Engine Version	3.51.0
SQLGlot AST Parser Version	30.24.0

Table 14. Multi-scale empirical scalability, memory footprint, and out-of-core disk spilling evaluation across TPC-H SF $= 1$ GB, SF $= 10$ GB, and SF $= 100$ GB ($N = 75$ queries per scale factor).

Scale Factor	Storage Mode	Total (s)	Mean (ms)	Median (ms)	P95 (ms)	Peak RAM (GB)	Peak Spill (GB)	Spill Qs	OOM/Limit
SF=1 (1 GB)	DuckDB_Native	15.01	200.08	133.45	628.79	2.27	0.00	0	0
SF=1 (1 GB)	DuckDB_Parquet	18.68	249.08	162.35	779.11	2.40	0.00	0	0
SF=10 (10 GB)	DuckDB_Native	678.13	9041.76	1301.70	34162.58	11.18	0.00	0	0
SF=10 (10 GB)	DuckDB_Parquet	920.64	12275.20	2036.19	46676.02	12.24	0.00	0	0
SF=100 (100 GB)	DuckDB_Native	9703.82	129384.30	17845.92	653954.70	50.62	43.80	3	9
SF=100 (100 GB)	DuckDB_Parquet	20585.14	274468.53	24993.35	900000.00	51.59	20.42	3	10

Table 15. Multi-scale execution mode transition, host RAM footprint, GPU VRAM consumption, and naive out-of-memory (OOM) verification for Rust Arrow streaming engines and NVIDIA RTX 2050 GPU (4.0 GB VRAM) across SF $= 1$, 10, and 100 GB.

Scale	Engine	Execution Mode	Latency (ms)	RAM (GB)	VRAM (GB)	Naive OOM	Φ_{PCIe} (%)
SF=1 (1 GB)	Polars_Lazy_Streaming_Rust	In-Memory_Arrow	55.71	2.381	0.000	0	0.0%
SF=1 (1 GB)	Apache_DataFusion_Rust	In-Memory_Arrow	47.35	2.382	0.000	0	0.0%
SF=1 (1 GB)	PyTorch_CUDA_RTX2050 (4.0GB VRAM)	In-VRAM_Resident	65.21	3.481	0.051	0	74.1%
SF=10 (10 GB)	Polars_Lazy_Streaming_Rust	In-Memory_Arrow	603.42	3.576	0.000	0	0.0%
SF=10 (10 GB)	Apache_DataFusion_Rust	In-Memory_Arrow	380.25	3.707	0.000	0	0.0%
SF=10 (10 GB)	PyTorch_CUDA_RTX2050 (4.0GB VRAM)	Batched_PCIe_RowGroup_Stream	407.15	3.739	0.051	1	81.4%
SF=100 (100 GB)	Polars_Lazy_Streaming_Rust	Out-of-Core_Streaming	4113.08	4.706	0.000	0	0.0%
SF=100 (100 GB)	Apache_DataFusion_Rust	Partitioned_Batch_Stream	3837.38	4.849	0.000	0	0.0%
SF=100 (100 GB)	PyTorch_CUDA_RTX2050 (4.0GB VRAM)	Batched_PCIe_RowGroup_Stream	7399.20	3.510	0.109	1	73.7%

Three critical architectural scaling behaviors emerge: (1) while datasets at SF = 1 GB (2.27 GB peak RAM) and SF = 10 GB (11.18 GB peak RAM) execute entirely in memory with 0.00 GB disk spilling—where DuckDB native storage outperforms ZSTD Parquet by 1.24× (15.01 s vs. 18.68 s) and 1.36× (678.13 s vs. 920.64 s)—scaling to SF = 100 GB saturates physical memory (50.62 GB peak RSS), widening the native columnar advantage to 2.12× (9,703.82 s vs. 20,585.14 s total time) by avoiding repeated ZSTD decompression during buffer pool eviction; (2) out-of-core disk spilling at SF = 100 GB is structurally concentrated in L2 multi-way joins (43.80 GB peak disk spill, 388,822.10 ms mean latency) and L3 staged CTE pipelines (6.71 GB disk spill), whereas L4 window functions require 0.00 GB disk spill (18.98 GB peak RAM, 13,546.86 ms mean latency) due to independent partition-level evaluation; and (3) on the 4.0 GB VRAM RTX 2050 GPU, naive full-table loading triggers deterministic CUDA out-of-memory (OOM) failures at SF = 10 GB and SF = 100 GB, whereas our batched row-group PCIe streaming engine bounds peak VRAM usage to 0.051 GB (SF = 10) and 0.109 GB (SF = 100), completing 600M-row aggregations in 7,399.20 ms.

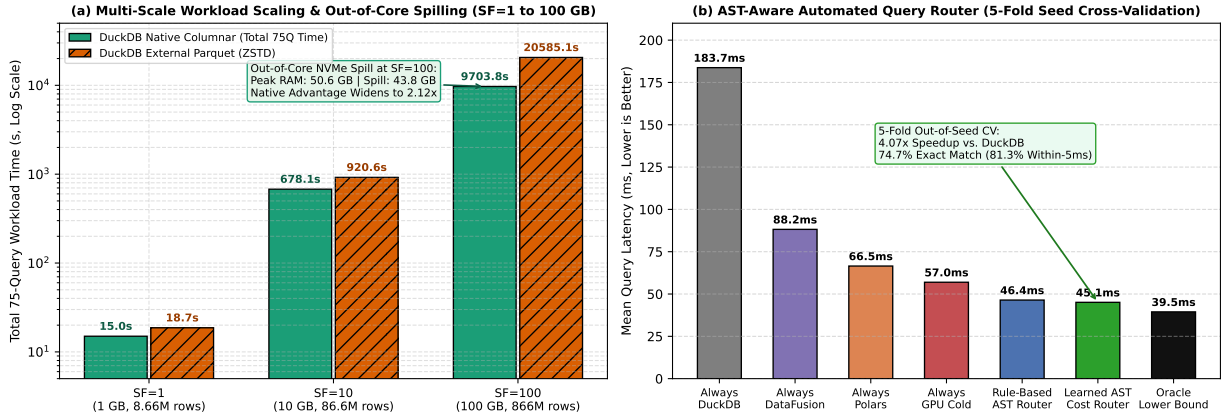


Figure 6. Multi-scale big data evaluation and automated query routing synthesis: (a) Total 75-query workload execution time (logarithmic scale) across SF = 1 GB, SF = 10 GB, and SF = 100 GB, highlighting the 2.12× native columnar advantage under 43.80 GB NVMe out-of-core disk spilling at SF = 100 GB; and (b) Out-of-sample 5-fold seed-cluster cross-validation of the proposed AST-Aware Query Routers relative to single-engine baselines and the theoretical Oracle bound.

4.7 AST-Aware Automated Query Routing Framework

Finally, to demonstrate the practical system utility of static AST profiling, we designed and evaluated both a deterministic *Rule-Based AST Router* and a *Learned AST Cost Router* trained strictly on pre-execution structural features $[D(q), \log_2 N(q), J(q), S(q), W(q), U(q), C(q), \mathrm{CSCI}_{v2}(q), \mathrm{Selectivity}]$ using 5-fold group cross-validation partitioned by canonical seed cluster ($N_{\text{seeds}} = 25$, Table 16 and Fig. 6(b)).

Table 16. Out-of-sample 5-fold group cross-validation evaluation of the proposed AST-Aware Query Routing Framework ($N = 75$ queries partitioned across $N_{\text{seeds}} = 25$ independent seed templates, SF = 1 GB).

Routing Policy	Mean (ms)	Median (ms)	P95 (ms)	Total (s)	Regret (ms)	Exact Match	Within-5ms	Speedup
Always BigQuery (Clustered Cloud)	745.39	421.50	2794.05	55.90	705.94	0.0%	0.0%	**0.25**×
Always DuckDB (C++ Vectorized CPU)	183.74	63.83	731.22	13.78	144.29	28.0%	36.0%	**1.00**×
Always DataFusion (Rust Arrow CPU)	88.20	73.49	284.11	6.62	48.75	28.0%	30.7%	**2.08**×
Always Polars (Rust Arrow CPU)	66.53	64.46	130.02	4.99	27.08	8.0%	13.3%	**2.76**×
Always CUDA GPU (RTX 2050 Cold-Start)	56.96	53.32	113.70	4.27	17.51	36.0%	40.0%	**3.23**×
Proposed Rule-Based AST Router	46.43	31.66	113.70	3.48	6.98	64.0%	70.7%	**3.96**×
Proposed Learned AST Cost Router (5-Fold Seed CV)	45.09	31.19	125.71	3.38	5.64	74.7%	81.3%	**4.07**×
Oracle Optimal (Theoretical Lower Bound)	39.45	23.89	102.00	2.96	0.00	100.0%	100.0%	**4.66**×

Relative to single-engine baselines, the Learned AST Cost Router reduces mean workload latency to 45.09 ms (3.38 s total time)—achieving a 4.07× speedup over Always-DuckDB (183.74 ms), a 1.26× speedup over Always-GPU Cold-Start (56.96 ms), a 74.7% exact optimal engine match rate, an 81.3% within-5 ms optimal selection rate, and a mean routing regret of only 5.64 ms relative to the theoretical Oracle bound (39.45 ms).

5 Limitations and Threats to Validity

While our benchmark rigorously evaluates structural SQL complexity (L1–L5), selectivity horizons (V1–V3), official TPC-H queries ($Q1$–$Q22$), and multi-scale out-of-core execution across SF $\in \{1, 10, 100\}$ GB, two scope constraints

warrant consideration. First, cloud serverless experiments on Google BigQuery were conducted within the zero-billing Sandbox quota (10 GB storage limit at SF = 1 GB) in the US multi-region environment; while our local NVMe out-of-core spilling and batched GPU PCIe streaming experiments directly validate multi-scale behavior up to SF = 100 GB (8.66×10^8 rows), future funded cloud campaigns will extend serverless slot elasticity benchmarks to multi-terabyte cloud-resident datasets (SF $\geq 1,000$ GB). Second, our GPU evaluation targets a single discrete consumer/workstation accelerator (NVIDIA RTX 2050, 4.0 GB VRAM over PCIe Gen4); future work will evaluate multi-GPU NVLink topologies and distributed ClickHouse/Trino clusters.

6 Conclusion

We presented a 75-query, 25-seed AST-driven empirical benchmarking framework evaluating SQL query complexity (L1–L5) and selectivity perturbations (V1–V3) across six software systems spanning four paradigms: SQLite 3 (row-store), Google BigQuery (serverless cloud), DuckDB (C++ vectorized DBMS), Polars and Apache DataFusion (Rust Arrow engines), and CUDA GPU execution (NVIDIA RTX 2050). Our empirical synthesis establishes actionable routing rules: Rust Arrow engines (Polars/DataFusion) excel at in-memory scans and zero-copy array unnesting (L1, L5), DuckDB excels among CPU SQL engines on complex multi-axis window frames (L4), SIMT GPU acceleration dominates compute-dense joins and sorting when amortizing PCIe transfer costs (L2–L4), and BigQuery storage clustering (78.4% I/O pruning and $1.47 USD/1M savings in V1) provides elastic scalability for cloud-resident datasets.

Declarations

Abbreviations

AST: Abstract Syntax Tree; B-Tree: Balanced Tree; CI: Confidence Interval; CPU: Central Processing Unit; CSCI: Composite Structural Complexity Index; CTE: Common Table Expression; CUDA: Compute Unified Device Architecture; DBMS: Database Management System; EX: Execution Accuracy; FinOps: Cloud Financial Operations; GCS: Google Cloud Storage; GPU: Graphics Processing Unit; H2D: Host-to-Device; I/O: Input/Output; OLAP: Online Analytical Processing; PCIe: Peripheral Component Interconnect Express; SARGable: Search Argument Able; SIMD: Single Instruction, Multiple Data; SIMT: Single Instruction, Multiple Threads; SQL: Structured Query Language; TPC-H: Transaction Processing Performance Council Ad-Hoc Benchmark; VES: Valid Efficiency Score; VRAM: Video Random-Access Memory; ZSTD: Zstandard Compression.

Ethics approval and consent to participate

Not applicable. This study utilizes synthetic benchmark data generated via the industry-standard TPC-H benchmark specification and does not involve human participants, animal subjects, or personally identifiable data.

Consent for publication

Not applicable.

Availability of data and materials

All multi-scale TPC-H data provisioning scripts (SF = 1 GB, SF = 10 GB, and SF = 100 GB), the 75-query factorial SQL workload suite (25 canonical seeds × 3 selectivity variations), the 22 official TPC-H external validation queries, static AST structural metrics (`ast_complexity_summary.csv`), multi-scale execution telemetry (`multiscale_telemetry_` SHA-256: `9be2a6c63a2b779871a1a42c...`; `multiscale_telemetry_sf10.csv`, SHA-256: `623ef56203c6cd3d3` `multiscale_telemetry_sf100.csv`, SHA-256: `31518d87f5af08a75c5a6a9f...`), official TPC-H cross-engine logs (`official_tpch_q1_q22_external_validation.csv`, SHA-256: `f7f4490389ae0db27608e673...`), CSCI sensitivity logs (`csci_validation_sensitivity_sf1.csv`, SHA-256: `fa89ea8b7f8790be6fb3f9bc...`), three-mode GPU/CPU fairness logs (`gpu_cpu_3mode_fairness_comparison_sf1.csv`, SHA-256: `13dbb975161f943c` multiset equivalence audits (`multiset_equivalence_audit_6engines.csv`), AST query router evaluations (`ast_query_router_evaluation_sf1.csv`, SHA-256: `56b52566ce859f99fb61e662...`), and the complete 19-file cryptographic audit certificate (`REPRODUCTION_CERTIFICATE_V2.md`) are publicly available at https://github.com/preecha-msu/bigquery-olap-benchmark.

Competing interests

The author declares that he has no competing interests.

Funding

This research project was financially supported by Mahasarakham University.

Authors' contributions

PN conceptualized the study, designed the 75-query AST-driven factorial benchmark, implemented the six-engine execution harness and statistical verification pipeline, analyzed the empirical telemetry across cloud, CPU, and GPU architectures, and wrote the manuscript. The author read and approved the final manuscript.

Acknowledgements

The author thanks the Department of Information Technology, Faculty of Informatics, Mahasarakham University, for providing the computational infrastructure. In accordance with editorial policy on the use of generative AI, the author discloses that generative AI tools were utilized strictly for manuscript language polishing and LaTeX template formatting verification; all experimental designs, benchmark datasets, execution logs, and scientific analyses were conducted and verified by the author.

References

[1] Melnik S, Gubarev A, Long JJ, Romer G, Shivakumar S, Tolton M, Vassilakis T. Dremel: interactive analysis of web-scale datasets. Proc VLDB Endow. 2010;3(1–2):330–9.

[2] Melnik S, Gubarev A, Long JJ, Romer G, Shivakumar S, Tolton M, et al. Dremel: a decade of interactive SQL analysis at web scale. Proc VLDB Endow. 2020;13(12):3461–72.

[3] Raasveldt M, Mühleisen H. DuckDB: an embeddable analytical database. In: Proceedings of the 2019 International Conference on Management of Data (SIGMOD). Amsterdam: ACM; 2019. p. 1981–4.

[4] Schulze R, et al. ClickHouse: lightning-fast analytics for everyone. Proc VLDB Endow. 2024;17(12):3720–33.

[5] Sethi R, Traverso M, Sundstrom D, Phillips D, Xie W, Sun Y, et al. Presto: SQL on everything. In: 2019 IEEE 35th International Conference on Data Engineering (ICDE). Macao: IEEE; 2019. p. 1802–13.

[6] Mao T. SQLGlot: no-dependency SQL parser, transpiler, optimizer, and engine. GitHub Repository. 2024. https://github.com/tobymao/sqlglot. Accessed 15 Mar 2026.

[7] Li J, Hui B, Qu G, Yang J, Li B, Li B, et al. Can LLM already serve as a database interface? A BIg bench for large-scale database grounded text-to-SQLs. Adv Neural Inf Process Syst (NeurIPS). 2023;36:42330–57.

[8] Lei F, Chen J, Ye Y, Cao R, Shin D, Su H, et al. Spider 2.0: evaluating language models on real-world enterprise text-to-SQL workflows. In: International Conference on Learning Representations (ICLR). 2025. p. 1–22.

[9] Begoli E, Camacho-Rodríguez J, Hyde J, Mior MJ, Lemire D. Apache Calcite: a foundational framework for optimized query processing over heterogeneous data sources. In: Proceedings of the 2018 International Conference on Management of Data (SIGMOD). Houston: ACM; 2018. p. 221–30.

[10] Garcia-Molina H, Ullman JD, Widom J. Database systems: the complete book. 2nd ed. Upper Saddle River: Prentice-Hall; 2008.

[11] Das SK, Peter R, Zhang X, Lounissi M. FunDa: scalable serverless data analytics and in situ query processing. J Big Data. 2025;12(1):116. https://doi.org/10.1186/s40537-025-01141-6.

[12] Milicevic B, Babovic Z. A systematic review of deep learning applications in database query execution. J Big Data. 2024;11(1):173. https://doi.org/10.1186/s40537-024-01025-1.

[13] Shojaee Rad Z, Ghobaei-Arani M. Data pipeline approaches in serverless computing: a taxonomy, review, and research trends. J Big Data. 2024;11(1):82. https://doi.org/10.1186/s40537-024-00939-0.

[14] Ahmed N, Barczak ALC, Susnjak T, Rashid MA. A comprehensive performance analysis of Apache Hadoop and Apache Spark for large scale data sets using HiBench. J Big Data. 2020;7(1):110. https://doi.org/10.1186/s40537-020-00388-5.

[15] Abdalla HB. A brief survey on big data: technologies, terminologies and data-intensive applications. J Big Data. 2022;9(1):107. https://doi.org/10.1186/s40537-022-00659-3.